

Lucrare de Laborator 3

Procesarea Semnalelor

Elaborat: Macrii Danu

Gr.: FAF-222

Chişinău, May 2025

Contents

Conclusions

15

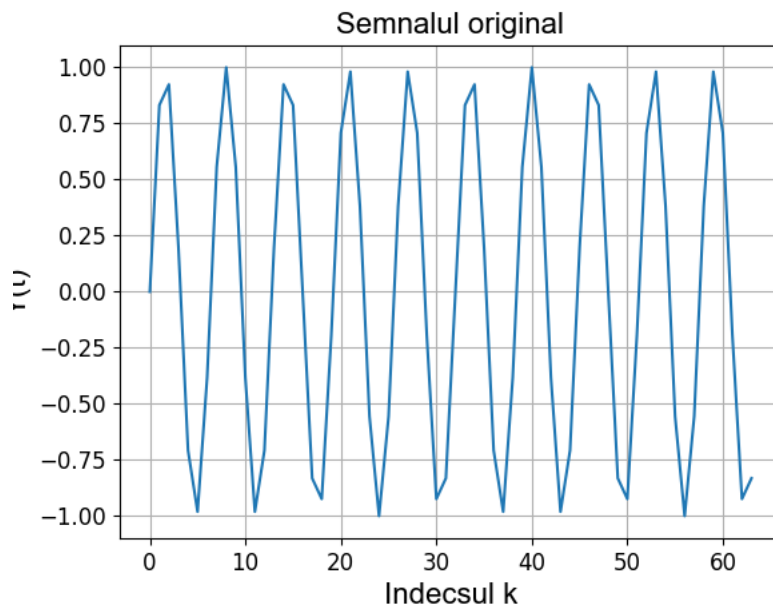
I. Transformarea Fourier

1.1 Generarea unui semnal discret

Condiții: Se generează un semnal sinusoidal cu $N = 64$ probe, frecvență de 20 Hz, eșantionat la 128 Hz. Se afișează semnalul original.

```
N = 64
T = 1 / 128 # Sampling period
k = np.arange(N)
f = np.sin(2 * np.pi * 20 * k * T) # Discrete signal

# Plot
plt.plot(k, f)
plt.grid(True)
plt.xlabel('Indecul k', fontsize=16, fontname='Arial')
plt.ylabel('Y(t)', fontsize=16, fontname='Arial')
plt.title('Semnalul original', fontsize=16, fontname='Arial')
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



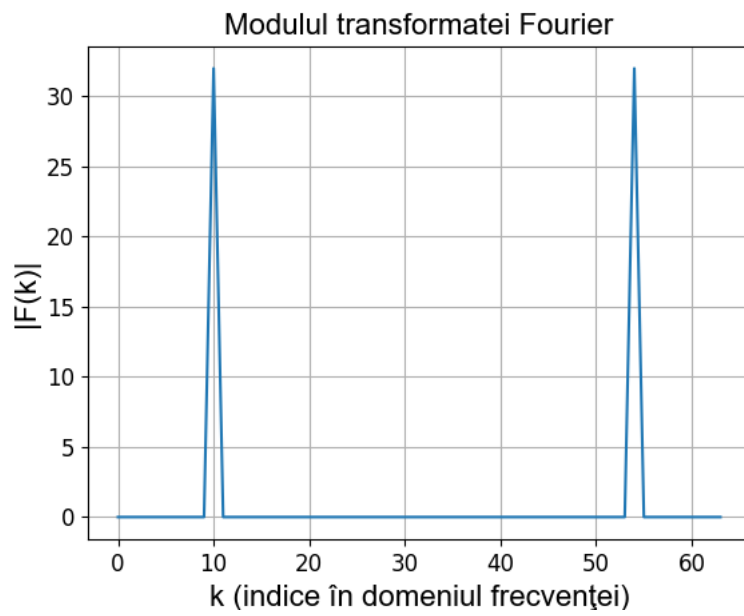
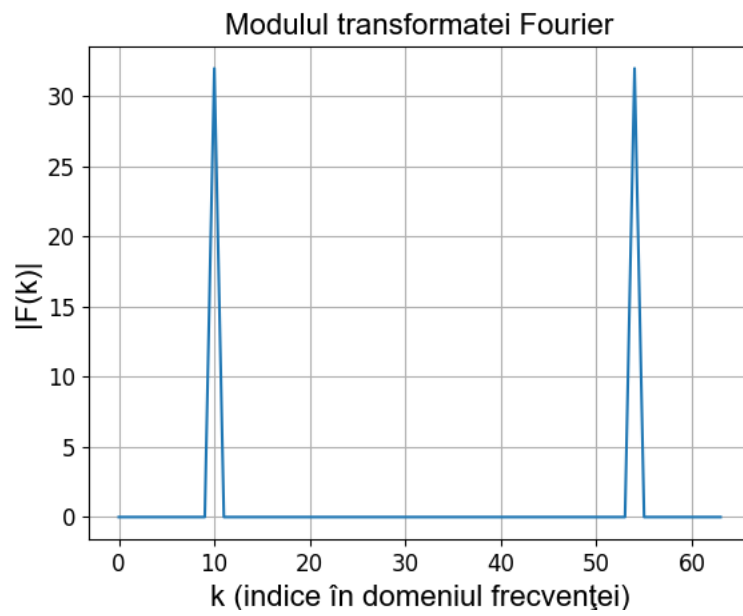
1.2 Transformata Fourier și localizarea frecvenței

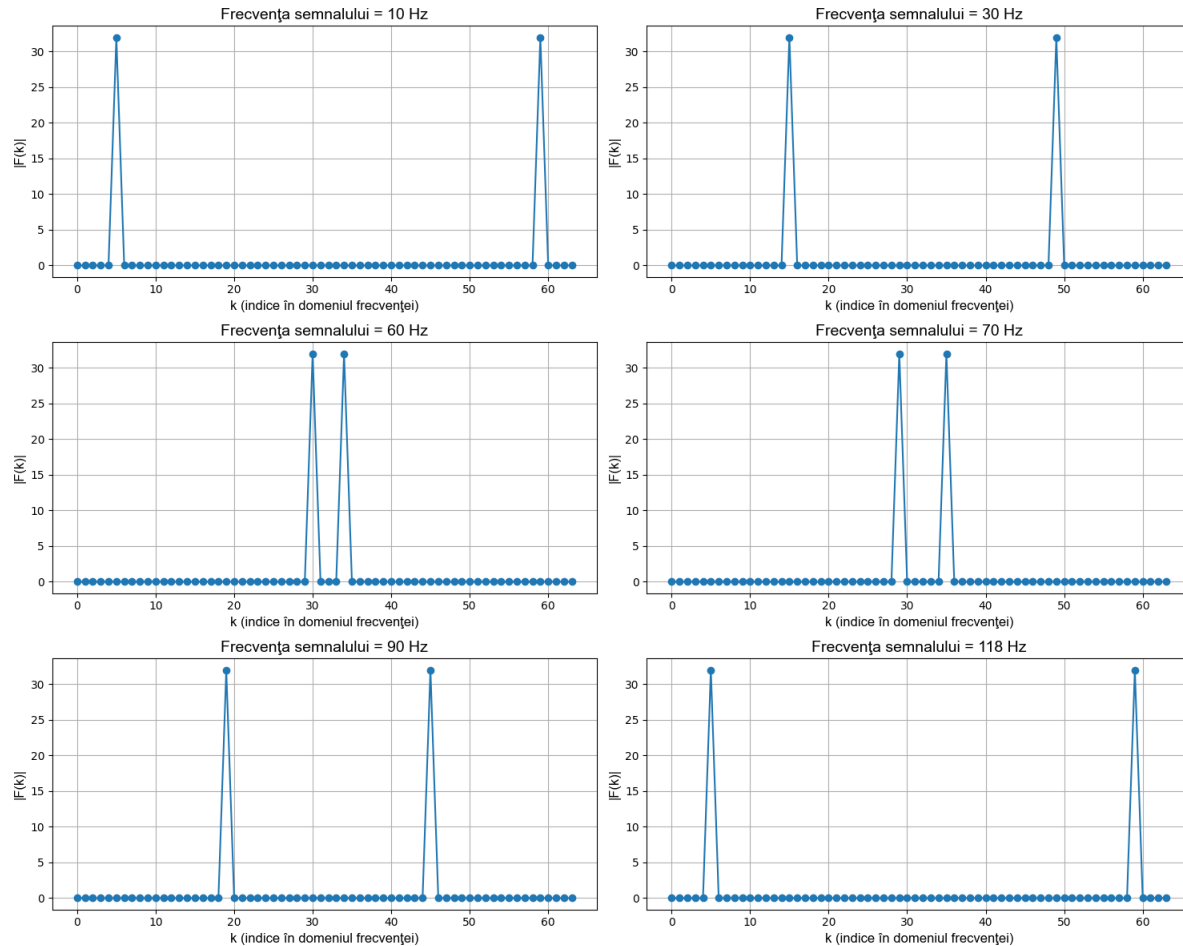
Condiții: Se aplică FFT pe semnalul generat, se afișează modulul transformatei Fourier. Se observă componenta la F_{10} și simetria la F_{54} . Se repetă pentru $f = 108$ Hz.

```
f = np.sin(2 * np.pi * 108 * k * T) # Original discrete signal

# Fourier Transform
F = np.fft.fft(f)
```

```
# Plot magnitude of FFT  
plt.plot(k, np.abs(F))
```





1.3 Spectru redus și axa în Hz

Condiții: Se afișează doar jumătate din spectru și se transformă axa x în Hz: $hertz = k \cdot (1/(N \cdot T))$.

```

N = 64
T = 1 / 128 # Sampling period
k = np.arange(N)
f = np.sin(2 * np.pi * 20 * k * T) # Example: 20 Hz sine wave

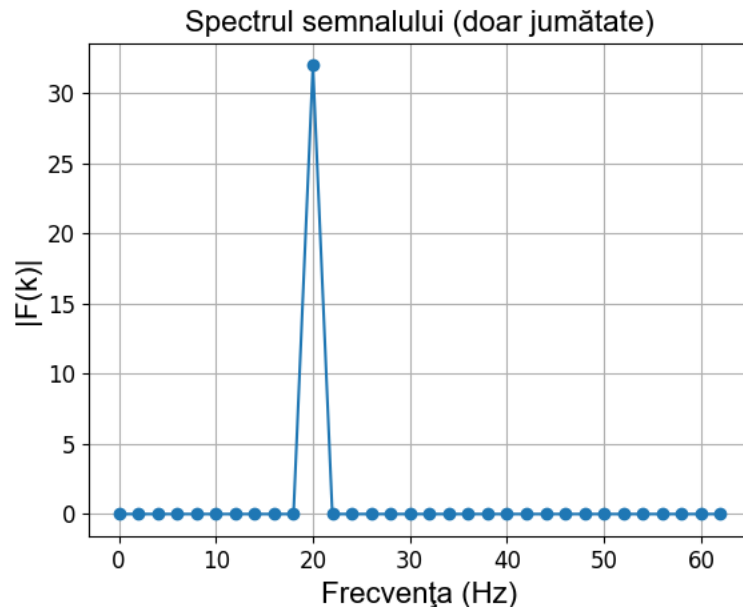
# Fourier Transform
F = np.fft.fft(f)
magF = np.abs(F)

# Frequency axis in Hz
hertz = k * (1 / (N * T)) # This gives freq = k * 2 Hz

# Plot only the first half of the spectrum
plt.plot(hertz[:N//2], magF[:N//2], marker='o')
plt.grid(True)
plt.xlabel('Frecvența (Hz)', fontsize=16, fontname='Arial')
plt.ylabel('|F(k)|', fontsize=16, fontname='Arial')
plt.title('Spectrul semnalului (doar jumătate)', fontsize=16, fontname='Arial')

```

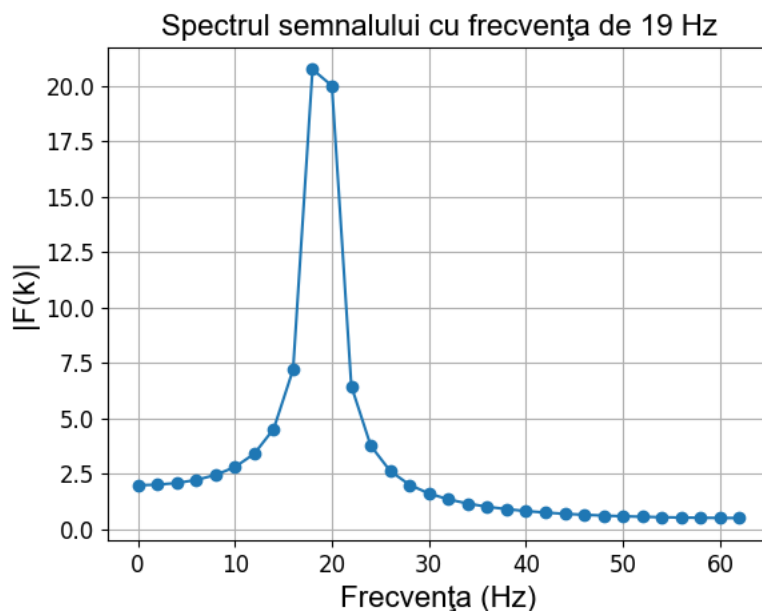
```
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



1.4 Frecvență nealiniată la pasul de 2Hz

Condiții: Se repetă punctul anterior cu $f = 19$ Hz. Se observă apariția energiei în jurul punctelor F_9 și F_{10} , nu exact în $F_{9.5}$.

```
f = np.sin(2 * np.pi * 19 * k * T) # Change the frequency to 19 Hz
```



II. Modulul și faza transformatei Fourier

2.1 Două sinusoide

Condiții: Se generează un semnal cu două componente: 15 Hz și 40 Hz. Se calculează FFT, se afișează magnitudinea și faza. Se marchează axele X la frecvențele 15, 40, 60, 85 Hz. Se repetă pentru alte 2-3 perechi de frecvențe.

```
N = 100 # Length of the signal
T = 1 / 99 # Sampling period
t = np.arange(0, 1, T) # Time vector

# New frequencies
f1_new = 25 # Frequency of the first sinusoid
f2_new = 50 # Frequency of the second sinusoid

# Create the new signal: sum of two sinusoids
x_new = np.sin(2 * np.pi * f1_new * t) + np.sin(2 * np.pi * f2_new * t)

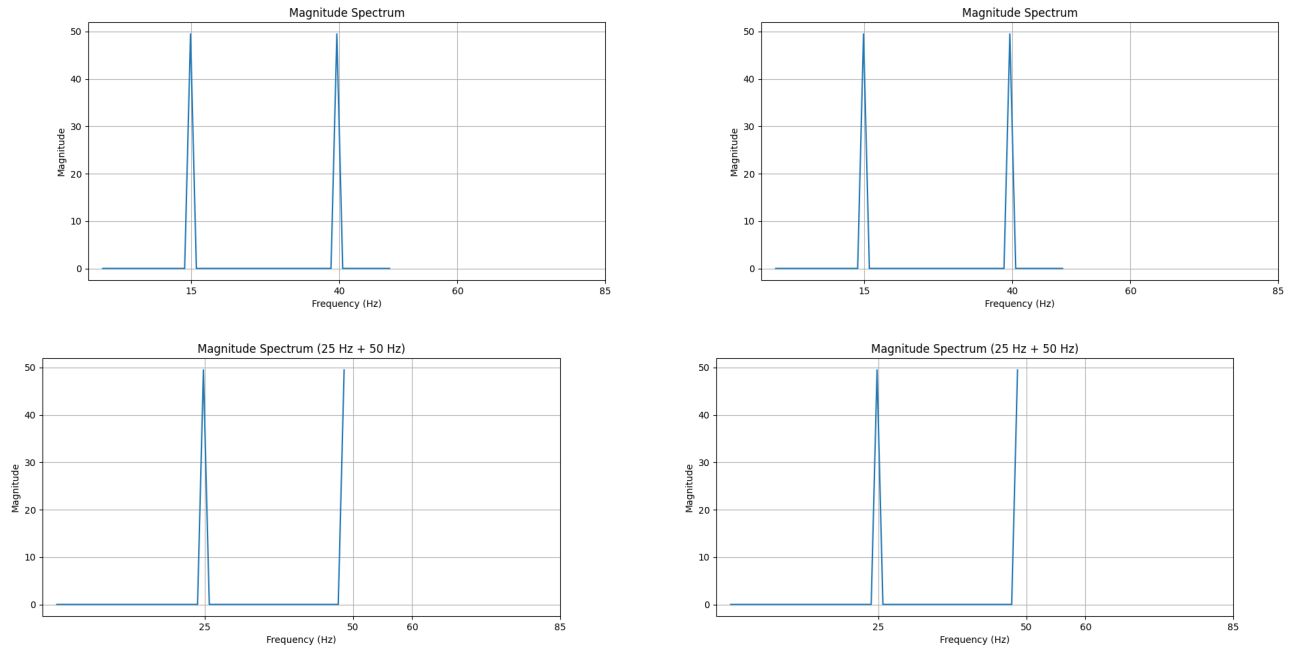
# Fourier Transform for the new signal
y_new = np.fft.fft(x_new)
m_new = np.abs(y_new) # Magnitude
p_new = np.unwrap(np.angle(y_new)) # Phase

f = np.fft.fftfreq(N, T) # Frequency axis

# Plot Magnitude
plt.figure(figsize=(10, 5))
plt.plot(f[:N // 2], m_new[:N // 2]) # Only plot the first half (positive frequencies)
plt.title('Magnitude Spectrum (25 Hz + 50 Hz)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.xticks([f1_new, f2_new, 60, 85])
plt.grid(True)

# Plot Phase
plt.figure(figsize=(10, 5))
plt.plot(f[:N // 2], p_new[:N // 2] * 180 / np.pi) # Phase in degrees
plt.title('Phase Spectrum (25 Hz + 50 Hz)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Phase (degrees)')
plt.xticks([f1_new, f2_new, 60, 85])
plt.grid(True)

plt.show()
```



III. Transformata Fourier a impulsului dreptunghiular

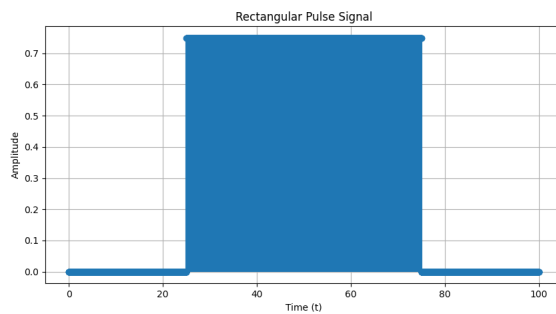
3.1 Semnal dreptunghiular

Condiții: Se creează un semnal dreptunghiular: $A = 0.75$, $w = 50$, $T_s = 0.01$, $T = 100$ Se utilizează funcții echivalente cu 'rectpuls' și se afișează semnalul.

```
A = 0.75 # Amplitude
w = 50   # Pulse width
Ts = 0.01 # Sampling period
T = 100  # Time duration

# Time vector
t = np.arange(0, T, Ts)

# Create a rectangular pulse
x = A * (np.abs(t % (2 * w) - w) < w / 2).astype(float)
```



3.2 Spectrul semnalului dreptunghiular

Condiții: Se aplică FFT și se afișează modulul cu axa $f = 0 : df : F_{max}$ Se repetă pentru $w = 5$ și $w = 0.5$.

```
A = 0.75 # Amplitude
Ts = 0.01 # Sampling period
T = 100 # Time duration

# Time vector
t = np.arange(0, T, Ts)

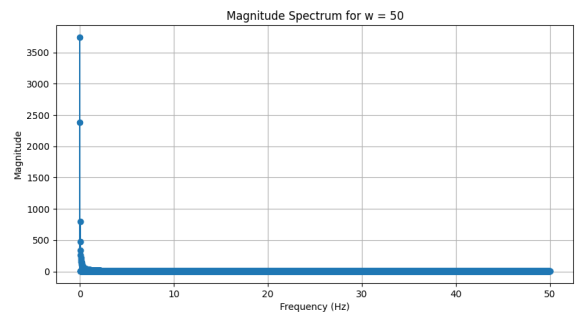
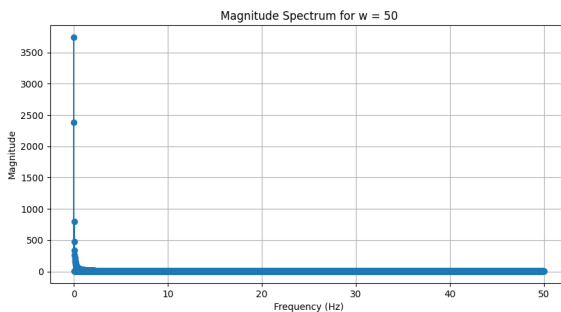
# Frequency parameters
df = 1 / T # Frequency resolution
Fmax = 1 / Ts # Maximum frequency
f = np.arange(0, Fmax, df) # Frequency vector

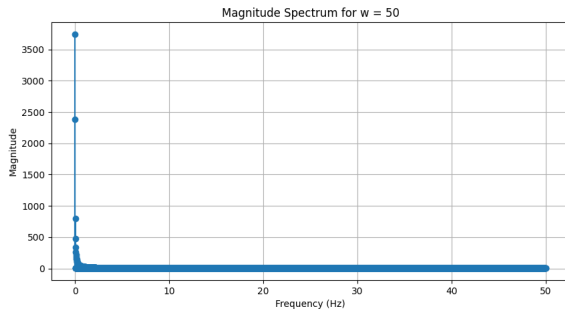
# Function to plot the FFT spectrum for a given width
def plot_spectrum(w):
    # Create a rectangular pulse with the current width w
    x = A * (np.abs(t % (2 * w) - w) < w / 2).astype(float)

    # Apply FFT
    y = np.fft.fft(x)

    # Plot the magnitude spectrum
    plt.figure(figsize=(10, 5))
    plt.stem(f[:len(y) // 2], np.abs(y)[:len(y) // 2], basefmt=" ")
    plt.title(f'Magnitude Spectrum for w = {w}')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.grid(True)
    plt.show()

# Repeat for different w values
for w_value in [50, 5, 0.5]:
    plot_spectrum(w_value)
```





3.3 Centrarea spectrului în jurul frecvenței 0

Condiții: Se aplică 'fftshift', se definește $f_1 = -F_{max}/2 : df : F_{max}/2$, se afișează modulul spectrului.

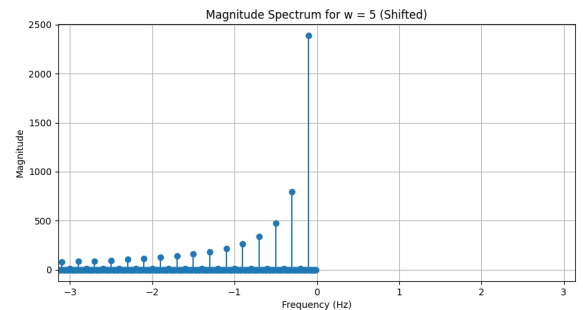
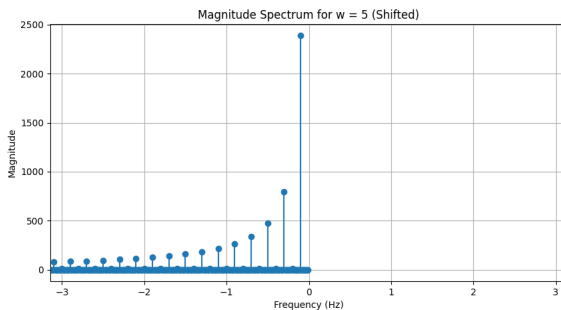
```
def plot_spectrum(w):
    # Create a rectangular pulse with the current width w
    x = A * (np.abs(t % (2 * w) - w) < w / 2).astype(float)

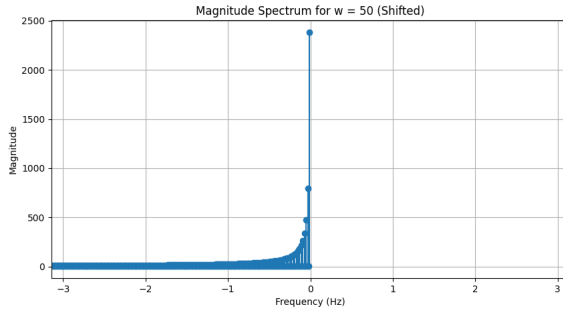
    # Apply FFT
    y = np.fft.fft(x)

    # Shift the FFT result using fftshift
    yp = np.fft.fftshift(y)

    # Frequency vector for the shifted FFT
    f1 = np.arange(-Fmax / 2, Fmax / 2, df)

    # Plot the magnitude spectrum
    plt.figure(figsize=(10, 5))
    plt.stem(f1[:len(yp) // 2], np.abs(yp)[:len(yp) // 2], basefmt=" ")
    plt.title(f'Magnitude Spectrum for w = {w} (Shifted)')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.grid(True)
    plt.xlim(-np.pi, np.pi) # Limiting x-axis to [-, ]
    plt.show()
```





3.4 Parte reală și imaginară

Condiții: Se afișează pe aceeași figură partea reală și partea imaginară a spectrului obținut.

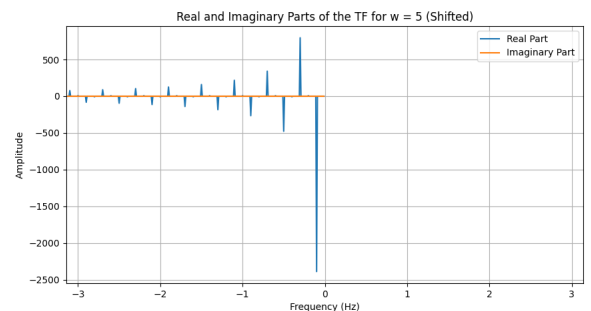
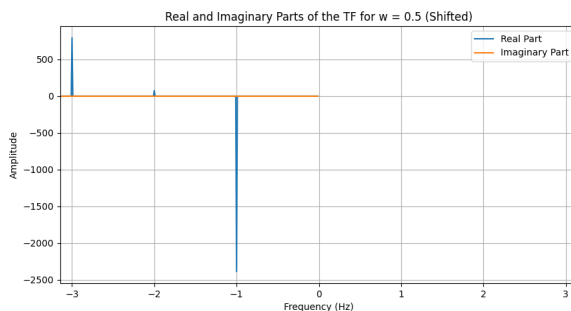
```
def plot_real_imag(w):
    # Create a rectangular pulse with the current width w
    x = A * (np.abs(t % (2 * w) - w) < w / 2).astype(float)

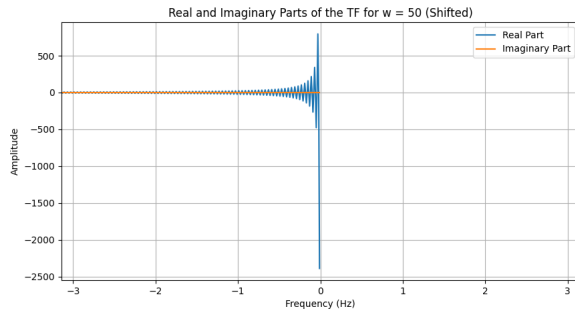
    # Apply FFT
    y = np.fft.fft(x)

    # Shift the FFT result using fftshift
    yp = np.fft.fftshift(y)

    # Frequency vector for the shifted FFT
    f1 = np.arange(-Fmax / 2, Fmax / 2, df)

    # Plot the real and imaginary parts
    plt.figure(figsize=(10, 5))
    plt.plot(f1[:len(yp) // 2], np.real(yp)[:len(yp) // 2], label='Real Part')
    plt.plot(f1[:len(yp) // 2], np.imag(yp)[:len(yp) // 2], label='Imaginary Part')
    plt.title(f'Real and Imaginary Parts of the TF for w = {w} (Shifted)')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Amplitude')
    plt.grid(True)
    plt.legend()
    plt.xlim(-np.pi, np.pi) # Limiting x-axis to [-, ]
    plt.show()
```





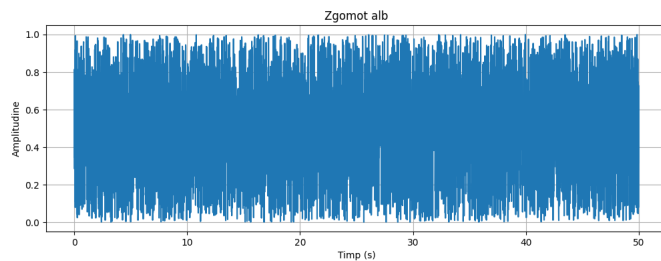
IV. TFD a zgomotului alb

4.1 Generare zgomot alb

Condiții: Se generează zgomot alb folosind funcția 'np.random.rand()'. $T_s = 0.01$, $T = 50$, $t = 0 : T_s : T$. Se afișează rezultatul.

```
Ts = 0.01
T = 50
t = np.arange(0, T + Ts, Ts)

x1 = np.random.rand(len(t))
```



4.2 Filtrarea zgomotului

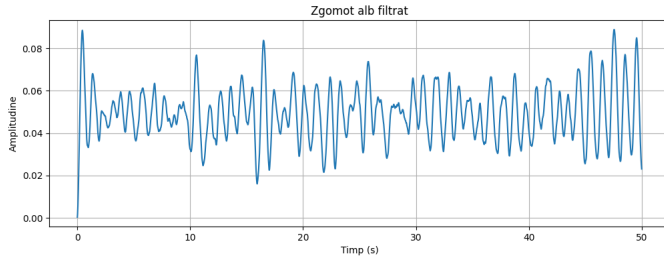
Condiții: Se filtrează zgomotul cu un filtru definit prin coeficienții a și b , unde $\omega_0 = 2\pi$, $dz = 0.05$, $A = 1$. Se afișează semnalul filtrat.

```
dz = 0.05
A = 1
om0 = 2 * np.pi
oms = om0 * Ts

a = [1 + 2 * dz * oms + oms**2,
     -2 * (1 + dz * oms),
     1]
b = [A * 2 * dz * oms**2]

b = b + [0, 0]

y1 = lfilter(b, a, x1)
```



4.3 Spectrul zgomotului înainte și după filtrare

Condiții: Se calculează și afișează spectrul semnalului înainte și după filtrare, folosind 'fftshift' și axe: $df = 1/T$, $F_{max} = 1/T_s$, $f = -F_{max}/2 : df : F_{max}/2$

```
Ts = 0.01
T = 50
t = np.arange(0, T + Ts, Ts)

x1 = np.random.rand(len(t))

dz = 0.05
A = 1
om0 = 2 * np.pi
oms = om0 * Ts

a = [1 + 2 * dz * oms + oms**2, -2 * (1 + dz * oms), 1]
b = [A * 2 * dz * oms**2, 0, 0] # completăm cu zerouri pentru ordinul a

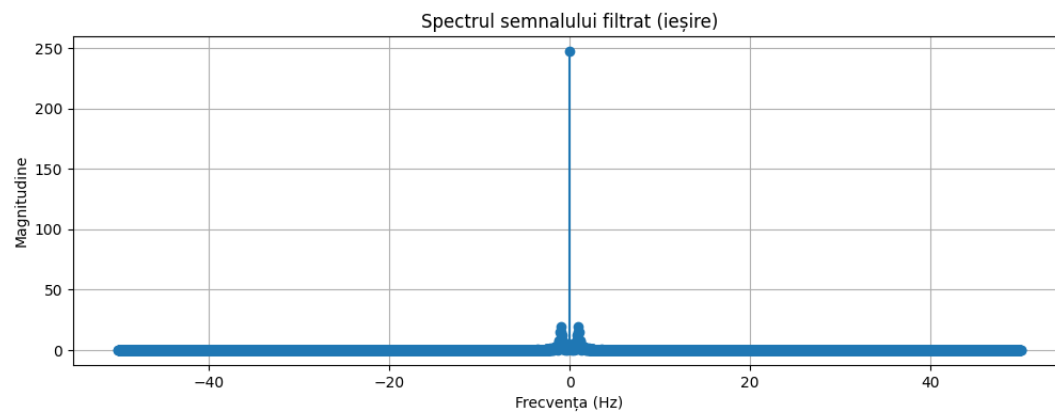
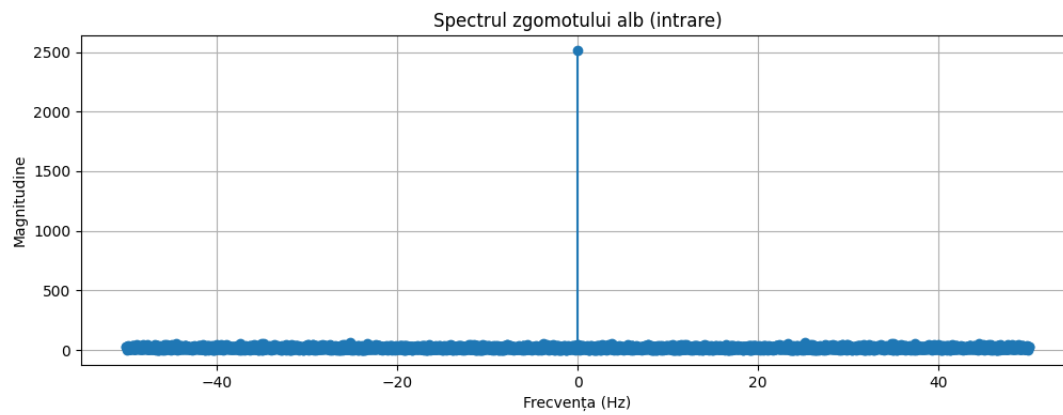
y1 = lfilter(b, a, x1)

Fu1 = np.fft.fft(x1)
Fu2 = np.fft.fft(y1)

Fu1p = np.fft.fftshift(Fu1)
Fu2p = np.fft.fftshift(Fu2)

m = np.abs(Fu1p)
m1 = np.abs(Fu2p)

df = 1 / T
Fmax = 1 / Ts
N = len(t)
f = np.linspace(-Fmax / 2, Fmax / 2, N)
```



Conclusions

In this laboratory work, I explored the Discrete Fourier Transform (DFT) using the FFT algorithm to analyze the frequency components of discrete-time signals. By generating sinusoidal and rectangular signals, we observed the spectral representation, energy distribution, and phase characteristics. We demonstrated the effect of signal frequency alignment with the sampling resolution and analyzed spectral leakage. Additionally, I investigated the Fourier transform behavior of white noise before and after linear filtering. The lab solidified understanding of spectral domain interpretation, frequency resolution, and the influence of signal parameters on DFT outcomes—key aspects in signal processing and system analysis.